

**TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
PHẦN HIỆU TẠI TP.HCM
BỘ MÔN CÔNG NGHỆ THÔNG TIN**



Kiểm thử Hộp đen — Các kỹ thuật thiết kế test case

Buổi 8 / 12 — Môn: Kiểm thử Phần mềm

Ngày 11 tháng 8 năm 2026

Nội dung buổi học

- 1 Kiểm thử hộp đen là gì?
- 2 Equivalence Partitioning (EP)
- 3 Boundary Value Analysis (BVA)
- 4 Thảo luận: hợp đồng API và validation
- 5 Decision Table Testing
- 6 State Transition Testing
- 7 Use Case Testing
- 8 Error Guessing & Exploratory
- 9 Chọn kỹ thuật nào, khi nào?
- 10 Thực hành trên DigiShield
- 11 Bài nộp (gắn với BTL)

Sợi chỉ đỏ

Toàn bộ ví dụ áp dụng trực tiếp lên các API và service THẬT của **DigiShield** mà nhóm đang kiểm thử trong BTL.

Vị trí trong đề cương môn học

Chương 5 — Kiểm thử hộp đen

Buổi 8 mở đầu Chương 5 của đề cương: *Giới thiệu — Khái niệm chung — Các phương pháp phân lớp* (phân lớp tương đương, giá trị biên, bảng quyết định, chuyển trạng thái, use case).

Buổi	Chương	Nội dung
3–4	C3	Hộp trắng: CFG, độ phủ, MC/DC, JaCoCo
5–7	C4	JUnit 5, Mockito, Integration test
8	C5	Hộp đen: EP, BVA, DT, ST, Use Case
9	C5	Khuôn mẫu test case; API testing: Postman + Newman
10–11	C6	Selenium E2E

Buổi 3–7 ta nhìn **vào trong** mã nguồn. Hôm nay ta lùi ra, chỉ nhìn **đặc tả** — đúng góc nhìn của tester và người dùng.

Ôn tập và động lực

Đã học (buổi 3–7)

- Độ phủ câu lệnh / nhánh / MC-DC
- JUnit 5 + Mockito, @SpringBootTest
- DigiShield có 284 test, JaCoCo đo độ phủ

Câu hỏi nhanh

Test evaluate() đạt 100% độ phủ nhánh thì đã đủ để kết luận “đúng nghiệp vụ” chưa?

Chưa!

Độ phủ chỉ nói mã **đã viết** được chạy qua. Nó không phát hiện:

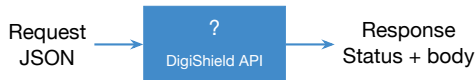
- Yêu cầu bị cài đặt **thiếu**
- Hành vi **khác đặc tả**
- Trường hợp đặc tả **bỏ sót**

Buổi này: kỹ thuật **có hệ thống** để chọn tập nhỏ nhất các test case có khả năng tìm lỗi cao nhất — từ đặc tả.

Kiểm thử hộp đen là gì?

Định nghĩa

Kiểm thử hộp đen (Black-box testing) là kỹ thuật kiểm thử trong đó test case được thiết kế **chỉ dựa trên đặc tả** (yêu cầu, tài liệu thiết kế, hợp đồng API), **không dựa vào mã nguồn** bên trong.



Đặc điểm:

- Không cần biết cách cài đặt
- Phù hợp với tester / BA
- Kiểm thử từ góc nhìn người dùng
- Phát hiện lỗi yêu cầu, thiết kế

Hộp đen so với Hộp trắng

Tiêu chí	Hộp đen	Hộp trắng
Cơ sở thiết kế	Đặc tả, yêu cầu, OpenAPI	Mã nguồn, luồng điều khiển
Kiến thức cần có	Nghịệp vụ, API	Ngôn ngữ lập trình, thuật toán
Người thực hiện	Tester, BA	Developer, tester kỹ thuật
Phát hiện	Lỗi yêu cầu, logic nghiệp vụ	Lỗi cài đặt, nhánh chưa test
Thước đo	Độ phủ đặc tả (lớp, biên, rule)	Coverage (statement, branch)
Ví dụ DigiShield	POST /interventions/evaluate trả đúng quyết định?	Nhánh PAUSE/WARN/ALLOW đều được chạy?
Buổi học	Buổi 8–9 (buổi này)	Buổi 3–4

Lưu ý

Hai kỹ thuật **bổ sung** cho nhau, không thay thế. BTL yêu cầu cả hai.

“Đặc tả” của DigiShield nằm ở đâu?

Test hộp đen cần đặc tả — DigiShield có 4 nguồn

- 1 **OpenAPI spec:** docs/DigiShield_openapi.yaml — endpoint, request/response schema (buổi 9 import vào Postman)
- 2 **Javadoc / mô tả nghiệp vụ** trên interface service (InterceptionService, ReportingService, ...)
- 3 **User story** của sản phẩm: “Analyst xác nhận báo cáo phishing thì hệ thống tự động đào tạo lại người dùng rủi ro”
- 4 **Hành vi giao diện:** các trang React (SOC Inbox, Watchlist)

Thực tế phũ phàng

Đặc tả thường **không đầy đủ**. Khi thiết kế test hộp đen mà không trả lời được “expected là gì?” ⇒ bạn vừa tìm ra một **lỗ hổng đặc tả** — cũng giá trị như tìm ra bug.

Các kỹ thuật hộp đen trong ISTQB

Kỹ thuật dựa trên đặc tả

- 1 **Equivalence Partitioning** (EP)
- 2 **Boundary Value Analysis** (BVA)
- 3 **Decision Table** Testing
- 4 **State Transition** Testing
- 5 **Use Case** Testing
- 6 Pairwise / Combinatorial
- 7 Cause-Effect Graphing

Buổi này: 5 kỹ thuật đầu + 2 kỹ thuật kinh nghiệm

- EP + BVA: từng trường đầu vào
- Decision Table: nhiều điều kiện kết hợp
- State Transition: hệ thống có trạng thái
- Use Case: kịch bản người dùng
- Error Guessing, Exploratory: dựa trên kinh nghiệm

Tham khảo: ISTQB CTFL Syllabus; Myers, *The Art of Software Testing*, ch. 4 [3].

Equivalence Partitioning — Nguyên lý

Định nghĩa

EP chia miền giá trị đầu vào thành các **lớp tương đương** (equivalence class) sao cho các giá trị trong cùng một lớp được hệ thống **xử lý theo cùng một cách**.

Nguyên tắc chọn đại diện

Nếu một giá trị trong lớp phát hiện lỗi, thì các giá trị khác trong cùng lớp cũng sẽ phát hiện lỗi đó.

⇒ Chỉ cần kiểm thử **một đại diện** từ mỗi lớp.

Phân loại lớp:

- **Hợp lệ:** hệ thống chấp nhận
- **Không hợp lệ:** hệ thống từ chối / xử lý riêng

Lợi ích:

- Giảm số test case
- Bao phủ đại diện mọi tình huống
- Dễ lý giải và review

EP — Quy trình áp dụng

- 1 **Xác định các trường đầu vào** cần kiểm thử.
- 2 **Với mỗi trường**, xác định tất cả ràng buộc từ đặc tả: kiểu dữ liệu, phạm vi (min/max), định dạng, bắt buộc/tùy chọn.
- 3 **Phân lớp** thành lớp hợp lệ và không hợp lệ theo từng ràng buộc.
- 4 **Chọn một đại diện** từ mỗi lớp.
- 5 **Kết hợp** đại diện từ nhiều trường thành test case hoàn chỉnh.

Áp dụng ngay trên DigiShield

Ba ví dụ: (a) nhãn tuổi báo cáo `ageLabel`, (b) tham số `size` của `GET /interventions`, (c) trường `action` của `POST /sim/events`.

Ví dụ (a): nhãn tuổi báo cáo phishing

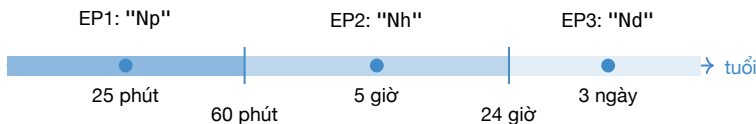
Đặc tả (từ javadoc ReportingServiceImpl)

Mỗi báo cáo trong SOC Inbox hiển thị **nhãn tuổi** gọn: dưới 60 phút → "Np" (phút); từ 1 giờ đến dưới 24 giờ → "Nh"; từ 24 giờ trở lên → "Nd".

```
// ReportingServiceImpl.java (module reporting)
private String ageLabel(Instant reportedAt, Instant now) {
    if (reportedAt == null) return null;
    Duration d = Duration.between(reportedAt, now);
    long mins = Math.max(0, d.toMinutes());
    if (mins < 60) return mins + "p"; // duoi 1 gio
    long hours = d.toHours();
    if (hours < 24) return hours + "h"; // duoi 1 ngay
    return d.toDays() + "d";          // tu 1 ngay
}
```

Tester hộp đen **không nhìn** code này — chỉ nhìn đặc tả. Ta chiếu code để đối chiếu sau khi phân lớp.

EP cho ageLabel — ba lớp thời gian



Lớp	Điều kiện (tuổi t)	Đại diện	Expected
EP1 (hợp lệ)	$0 \leq t < 60$ phút	$t = 25$ phút	"25p"
EP2 (hợp lệ)	$60 \text{ phút} \leq t < 24$ giờ	$t = 5$ giờ	"5h"
EP3 (hợp lệ)	$t \geq 24$ giờ	$t = 3$ ngày	"3d"

Mỗi lớp một đại diện $\Rightarrow$ 3 test case thay vì vô hạn giá trị thời gian.

EP cho ageLabel — các lớp “khó chịu”

Đặc tả im lặng về hai tình huống

- `reportedAt = null` (dữ liệu cũ, import lỗi)?
- `reportedAt` **sau** `now` (lệch đồng hồ giữa client/server, múi giờ sai)?

Lớp	Điều kiện	Hành vi thực tế	Câu hỏi cho spec
EP4	<code>reportedAt = null</code>	trả về <code>null</code>	UI hiển thị gì khi nhận <code>null</code> ?
EP5	báo cáo “tù tương lai”	<code>Math.max(0, ...)</code> $\rightarrow$ “0p”	che giấu lỗi lệch giờ có ổn không?

Bài học

EP nghiêm túc luôn hỏi: “*còn lớp đầu vào nào mà đặc tả chưa nói đến?*” Lớp không hợp lệ / bất thường thường là nơi tìm ra defect và lỗi hổng đặc tả.

Ví dụ (b): tham số size của GET /interventions

Đặc tả

GET /api/v1/interventions?page=&size= (role ANALYST) trả về danh sách sự kiện can thiệp, phân trang; size là số phần tử mỗi trang, hợp lệ từ 1 đến 100.

```
// InterceptionServiceImpl.listInterventions(page, size)
int safePage = Math.max(page, 1);
int safeSize = Math.min(Math.max(size, 1), 100);
Pageable pageable = PageRequest.of(safePage - 1, safeSize);
```

Lớp	Điều kiện	Đại diện	Hành vi thực tế
EP1 (không hợp lệ)	$size < 1$	0, -5	clamp về 1, HTTP 200
EP2 (hợp lệ)	$1 \leq size \leq 100$	20	dùng nguyên giá trị, 200
EP3 (không hợp lệ)	$size > 100$	500	clamp về 100, HTTP 200

Lớp “không hợp lệ” không bị từ chối mà bị **sửa lạng lế** — ta sẽ thảo luận ở phần BVA.

Ví dụ (c): trường action của POST /sim/events

Đặc tả

POST /api/v1/sim/events với body {campaignId, userId, action} ghi nhận tương tác của người học với chiến dịch mô phỏng. action là một trong 5 giá trị enum: DELIVERED, OPEN, CLICK, SUBMIT, REPORT.

Lớp	Điều kiện	Đại diện	Expected
EP1 (hợp lệ)	đúng 1 trong 5 enum	"CLICK"	200, event lưu
EP2 (không hợp lệ)	chuỗi ngoài enum	"HACKED"	400
EP3 (không hợp lệ)	sai chữ hoa/thường	"click"	? — spec chưa nói
EP4 (không hợp lệ)	null / thiếu trường	(bỏ action)	400

Nhận xét

Với trường kiểu **enum**, mỗi giá trị hợp lệ là một lớp con đáng test (5 giá trị → 5 TC) vì mỗi action đi vào thống kê chiến dịch khác nhau.

EP — Lưu ý khi kết hợp nhiều trường

Giả định một lỗi (single-fault assumption)

Để test lớp **không hợp lệ** của một trường, giữ các trường khác ở giá trị **hợp lệ**. Nếu hai trường cùng sai, không rõ lỗi do trường nào gây ra.

Ví dụ với POST /sim/events

- Test `action="HACKED"` $\Rightarrow$ `campaignId`, `userId` phải là UUID có thật trong dữ liệu demo.
- Test `campaignId` không tồn tại $\Rightarrow$ `action="CLICK"` hợp lệ.
- Các lớp **hợp lệ** của nhiều trường có thể gộp vào chung một test case để tiết kiệm.

Boundary Value Analysis — Nguyên lý

Động lực

Kinh nghiệm cho thấy lỗi **thường xuất hiện tại vùng biên** giữa các lớp tương đương hơn là ở giữa lớp. Lập trình viên dễ viết sai `<` thành `<=`, `Math.max` thành `Math.min`.

BVA 2-value

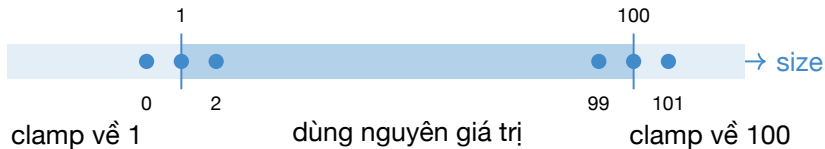
Với mỗi biên, kiểm thử **hai** giá trị: ngay tại biên (*on-point*) và sát biên phía bên kia (*off-point*).

BVA 3-value

Kiểm thử **ba** giá trị cho mỗi biên: *below-point*, *on-point*, *above-point*.

Cả hai biến thể đều thuộc ISTQB CTFL Syllabus v4.0 (2023); trước v4.0, 3-value nằm ở cấp Advanced.

BVA — size từ 1 đến 100 (GET /interventions)



TC	size	Loại điểm	safeSize thực tế	Expected (spec?)
TC-B1	0	off-point biên dưới	1	400 hay 200-với-1?
TC-B2	1	on-point biên dưới	1	200, 1 phần tử/trang
TC-B3	2	above-point	2	200, 2 phần tử/trang
TC-B4	99	below-point	99	200
TC-B5	100	on-point biên trên	100	200, tối đa 100
TC-B6	101	off-point biên trên	100	400 hay 200-với-100?

Thảo luận: clamp lặng lẽ — feature hay bug?

Quan sát từ BVA

`size=0` và `size=101` **không gây lỗi 400**: code `Math.min(Math.max(size, 1), 100)` lặng lẽ đưa về 1 và 100.

Quan điểm “feature”:

- API bao dung, khó bị phá bằng tham số phân trang
- Trải nghiệm client tốt hơn (không chết trang)

Quan điểm “bug tiềm ẩn”:

- Hành vi này **không có trong spec** $\Rightarrow$ client không biết
- Che giấu lỗi phía client (gửi `size=-1` vẫn “chạy”)

Kết luận cho tester

Test hộp đen **không tự quyết** đúng/sai — nó buộc đội dự án **chốt spec**: hoặc ghi rõ “ngoài khoảng thì clamp” vào OpenAPI, hoặc đổi code trả 400. Expected của TC-B1/TC-B6 chỉ viết được sau khi chốt.

BVA — điểm rủi ro 0..100 (module analytics)

Đặc tả AnalyticsServiceImpl.computeScore

Điểm rủi ro người dùng = điểm nền 5 cộng trọng số tín hiệu (click mô phỏng, báo cáo xác nhận...trong cửa sổ 90 ngày), **chặn trong đoạn** $[0, 100]$:
 $\text{Math.max}(0, \text{Math.min}(100, 5 + \text{signalWeight}))$.

TC	signalWeight	score thô	Expected
TC-R1	0 (không tín hiệu)	5	5 (điểm nền)
TC-R2	94	99	99
TC-R3	95	100	100 (on-point)
TC-R4	96	101	100 (bị chặn trên)
TC-R5	-5	0	0 (on-point dưới)
TC-R6	-6	-1	0 (bị chặn dưới)

Biên của **đầu ra** cũng cần BVA, không chỉ đầu vào: dashboard hiển thị sai khi score vượt 100 là lỗi nghiêm trọng về niềm tin.

BVA — độ tin cậy AI trong StubAiClient

Đặc tả

POST /ai/classify: nội dung có tín hiệu lừa đảo → nhãn "threat" với $\text{confidence} = 0.6 + 0.1 \times (\text{số tín hiệu})$, chặn tối đa 1.0.

```
// StubAiClient.classify(payload)
long threatHits = countHits(text, THREAT_SIGNALS);
if (threatHits > 0) {
    double confidence = clamp(0.6 + 0.1 * threatHits);
    return new ClassificationView("threat", confidence, ...);
}
// clamp(v) = Math.max(0.0, Math.min(1.0, v))
```

Số tín hiệu	1	4	5
confidence	0.7	1.0 (chạm trần)	1.0 (bão hòa)

Biên nằm ở **4 tín hiệu**: từ đó trở đi confidence không tăng nữa. Test data: payload chứa "otp", "urgent", "login", ...

EP + BVA kết hợp — bộ test cho ageLabel

TC-ID	Tuổi báo cáo	Kỹ thuật	Lớp/biên	Expected
TC-A1	0 phút	BVA	on-point biên dưới	"0p"
TC-A2	25 phút	EP	EP1 đại diện	"25p"
TC-A3	59 phút	BVA	below 60 phút	"59p"
TC-A4	60 phút	BVA	on-point 60 phút	"1h"
TC-A5	5 giờ	EP	EP2 đại diện	"5h"
TC-A6	23 giờ 59 phút	BVA	below 24 giờ	"23h"
TC-A7	24 giờ	BVA	on-point 24 giờ	"1d"
TC-A8	3 ngày	EP	EP3 đại diện	"3d"
TC-A9	null	EP	EP4 bất thường	null (spec?)
TC-A10	-10 phút (tương lai)	EP	EP5 bất thường	"0p" (spec?)

Buổi 5–7 đã học cách cài đặt

Bảng này chuyển thẳng thành `@ParameterizedTest` với `@CsvSource` — hàm thuần `ageLabel` là ứng viên hoàn hảo.

EP và BVA — Tổng kết so sánh

Thuộc tính	EP	BVA
Mục tiêu	Giảm số test case bằng cách chọn đại diện	Tập trung vào điểm dễ lỗi nhất
Áp dụng khi	Đầu vào chia được thành vùng xử lý khác nhau	Có phạm vi / giới hạn rõ ràng
Số test case	Bằng số lớp	Gấp đôi (hoặc ba) số biên
Loại lỗi	Xử lý sai theo nhóm	Off-by-one, điều kiện biên sai
DigiShield	action enum, 3 lớp ageLabel	size 1..100, score 0..100
Kết hợp	Luôn dùng cả hai; BVA là EP được tinh chỉnh tại biên	

DigiShield không dùng Bean Validation

Quan sát kiến trúc

DigiShield **không** dùng `@NotNull`, `@Email`, `@Size` trên DTO. Mọi kiểm tra đầu vào viết **thủ công trong service**:

```
// AuthServiceImpl.createUser(input)
String email = input.email();
if (email == null || email.isBlank()) {
    throw new IllegalArgumentException("email is required");
}
```

Hệ quả với tester:

- Không thể “đọc annotation ra spec”
- Mỗi trường phải **thủ mới biết** hành vi thật

Hệ quả với hệ thống:

- `IllegalArgumentException` không được handler nào dịch sang HTTP status chuẩn
- Kết quả trả về client là gì? → slide sau

Thí nghiệm: POST /users với email rỗng — 400 hay 500?

Làm ngay trên máy (dev profile, không cần Docker)

Gửi POST /api/v1/users với body {"email": "", "displayName": "Test"}
(profile dev: DevSecurityConfig permitAll — không cần token). Quan sát status code.

Kỳ vọng theo hợp đồng API

Lỗi **của client** ⇒ 400 Bad Request +
thông báo chỉ rõ trường sai.

Hành vi thực tế

IllegalArgumentException nổ ra
khỏi service, không có
@ControllerAdvice ⇒ Spring trả
500 Internal Server Error.

Bài học về hợp đồng API

500 nghĩa là “lỗi của server” — client sẽ retry, alert sẽ réo. Một test hộp đen tốt **phân biệt 4xx với 5xx**: đây là defect thật tìm được chỉ bằng EP lớp không hợp lệ, không cần đọc code. (Định dạng email sai kiểu "abc" thì sao? — không có kiểm tra nào cả!)

Decision Table Testing — Nguyên lý

Khi nào dùng?

Khi kết quả đầu ra phụ thuộc vào **tổ hợp** của nhiều điều kiện đầu vào. Một điều kiện đơn lẻ không đủ để xác định hành vi hệ thống.

Cấu trúc một Decision Table:

- **Conditions** (hàng trên): điều kiện đầu vào
- **Actions** (hàng dưới): hành động / kết quả
- **Rules** (cột): mỗi cột một tổ hợp điều kiện
- n điều kiện Boolean $\Rightarrow$ tối đa 2^n rules

Ký hiệu

T	Điều kiện đúng
F	Điều kiện sai
–	Không quan trọng
✓	Hành động xảy ra
(trống)	Không xảy ra

Ví dụ chủ lực: can thiệp giao dịch đáng ngờ

Đặc tả POST /interventions/evaluate

Khi người dùng chuyển tiền, hệ thống chấm 3 tín hiệu: đang có **cuộc gọi** (onCall), **người nhận mới** (newPayee), tài khoản đích **trong watchlist** (watchlistHit). Cả 3 → PAUSE; chỉ cần watchlist → WARN; còn lại ALLOW.

```
// InterceptionServiceImpl.evaluate(request)
if (request.onCall() && request.newPayee()
    && watchlistHit) {
    decision = Decision.PAUSE;    // chặn + giao duc
} else if (watchlistHit) {
    decision = Decision.WARN;    // canh bao
} else {
    decision = Decision.ALLOW;   // cho qua
}
eventRepository.save(event);    // lon ghi log
```

Decision Table đầy đủ: $2^3 = 8$ rules

	R1	R2	R3	R4	R5	R6	R7	R8
Điều kiện								
onCall	T	T	T	T	F	F	F	F
newPayee	T	T	F	F	T	T	F	F
watchlistHit	T	F	T	F	T	F	T	F
Hành động								
PAUSE	✓							
WARN			✓		✓		✓	
ALLOW		✓		✓		✓		✓
Lưu InterventionEvent	✓	✓	✓	✓	✓	✓	✓	✓

Mỗi cột là một test case tiềm năng. Hàng “Lưu InterventionEvent” nhắc ta: **hành động phụ** (ghi log, publish event) cũng phải kiểm tra, không chỉ giá trị trả về.

Rút gọn Decision Table

Ba luật nghiệp vụ, bốn cột rút gọn

(1) PAUSE khi **cả ba** tín hiệu; (2) WARN khi watchlistHit nhưng **không đủ cả ba**; (3) ALLOW khi **không** watchlistHit.

	R1	R2'	R3'	R4'
onCall	T	F	T	-
newPayee	T	-	F	-
watchlistHit	T	T	T	F
PAUSE	✓			
WARN		✓	✓	
ALLOW				✓

R4' gộp R2, R4, R6, R8 (không watchlist → luôn ALLOW). Luật WARN “watchlistHit và không đủ cả ba” cần **hai** cột R2'/R3' vì ký hiệu – không diễn đạt được “không đồng thời cả hai T” trong một cột — 8 rules rút còn 4, vẫn phủ đủ 3 hành vi.

Decision Table phát hiện tổ hợp đáng ngờ

Nhìn lại cột R2 của bảng đầy đủ

onCall = T, newPayee = T, watchlistHit = F $\Rightarrow$ **ALLOW?**!

Người dùng **đang nghe điện thoại** và chuyển tiền cho **người nhận hoàn toàn mới** — kịch bản lừa đảo qua cuộc gọi kinh điển ở VN — nhưng vì tài khoản đích chưa kịp vào watchlist, hệ thống cho qua không một lời cảnh báo.

Thảo luận cả lớp

- Đây là bug của **code** hay của **đặc tả**? (Code làm đúng spec!)
- Nếu là bạn, bạn đề xuất rule mới nào cho R2? (WARN?)
- Ai là người quyết định — tester, developer hay product owner?

Đây chính là sức mạnh của Decision Table: **liệt kê đủ tổ hợp** buộc ta đối diện những trường hợp mà đặc tả viết theo lối kể chuyện đã bỏ quên.

Từ rule thành test case API

TC-ID	Rule	Input (POST /interventions/evaluate)	Expected
TC-DT-1	R1	onCall=true, newPayee=true, destAccount có trong watchlist	decision "PAUSE", 3 signals
TC-DT-2	R2	onCall=true, newPayee=true, destAccount lạ	decision "ALLOW", 2 signals
TC-DT-3	R3'	onCall=true, newPayee=false, destAccount trong watchlist	decision "WARN"
TC-DT-4	R2'	onCall=false, newPayee=true, destAccount trong watchlist	decision "WARN"
TC-DT-5	R4'	onCall=false, newPayee=false, destAccount lạ	decision "ALLOW", 0 signals

Chuẩn bị dữ liệu

“destAccount có trong watchlist” là **precondition**: tạo trước bằng POST /account-watchlist (role ANALYST). Buổi 9 ta sẽ tự động hóa chuỗi này bằng Postman.

State Transition Testing — Nguyên lý

Định nghĩa

Kiểm thử hành vi của hệ thống khi **chuyển từ trạng thái này sang trạng thái khác** dưới tác động của các sự kiện (events).

Thành phần của máy trạng thái:

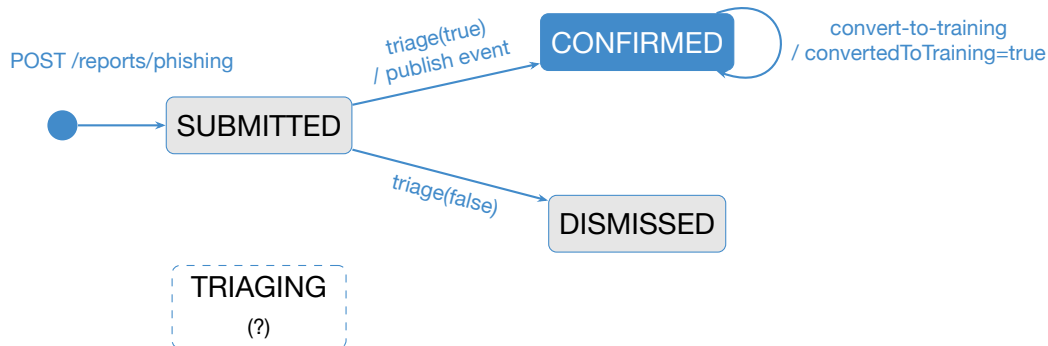
- **State:** trạng thái tại một thời điểm
- **Event:** sự kiện kích hoạt
- **Guard:** điều kiện phụ
- **Action:** hành động khi chuyển tiếp
- **Transition:** cạnh nối hai state

Ký hiệu:



Áp dụng khi: hành vi phụ thuộc vào trạng thái hiện tại — báo cáo phishing, mẫu AI, chiến dịch mô phỏng.

Vòng đời báo cáo phishing (PhishingReport)



Enum thật: `ReportStatus` = {`SUBMITTED`, `TRIAGING`, `CONFIRMED`, `DISMISSED`}. Khi `CONFIRMED`, service publish `PhishingReportConfirmedEvent` — kích hoạt pipeline AIDA (chấm lại rủi ro, tự ghi danh khóa học).

State Transition Table cho PhishingReport

Theo đặc tả kỳ vọng

Mỗi ô: trạng thái kế tiếp khi nhận event; --- = chuyển tiếp không hợp lệ, hệ thống phải từ chối.

State \ Event	triage(true)	triage(false)	convert-to-training
SUBMITTED	CONFIRMED	DISMISSED	—
CONFIRMED	—	—	CONFIRMED (+flag)
DISMISSED	—	—	—

Test case sinh ra

Ô có tên state $\Rightarrow$ test chuyển tiếp **hợp lệ** (kiểm cả action phụ: publish event, set aiLabel). Ô --- $\Rightarrow$ test chuyển tiếp **không hợp lệ**: gọi và kỳ vọng bị từ chối.

Điều mà State Transition Testing phát hiện ra

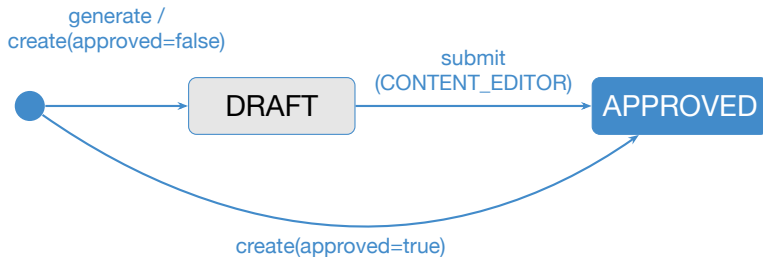
Hai phát hiện khi đối chiếu bảng với hệ thống thật

- 1 **Trạng thái chết** (dead state): TRIAGING có trong enum nhưng **không có chỗ nào trong code gán nó** — trạng thái không thể đạt tới. Spec thừa hay code thiếu?
- 2 **Không có guard trạng thái**: `triage()` không kiểm tra status hiện tại. Triage lại một báo cáo đã CONFIRMED bằng `decision=dismiss` ⇒ nó lặng lẽ thành DISMISSED (và ngược lại). `convert-to-training` cũng chạy được trên báo cáo DISMISSED!

Vì sao invalid transition đáng test đến vậy?

Người dùng thật sẽ bấm nút hai lần, gọi API sai thứ tự, replay request. Hệ thống nghiệp vụ an ninh (chống lừa đảo!) mà cho phép “lật” kết luận threat ↔ clean không kiểm soát là rủi ro kiểm toán nghiêm trọng.

Máy trạng thái thứ hai: mẫu phishing AI (AiTemplate)



- Mẫu do AI sinh (POST `/ai/templates/generate`) **luôn** vào DRAFT — con người phải duyệt trước khi dùng cho chiến dịch.
- Câu hỏi state-transition: mẫu APPROVED có “submit” lại được không? Chiến dịch có được phép chọn mẫu DRAFT không? → hai test không hợp lệ đáng giá.

Các chiến lược bao phủ chuyển trạng thái

All states

Mỗi **trạng thái** được đi qua ít nhất một lần — tiêu chí riêng, yếu nhất.

0-switch (all transitions)

Mỗi **chuyển tiếp hợp lệ** được thực thi ít nhất một lần — mức tối thiểu nên đạt.

1-switch

Mọi **cặp** chuyển tiếp liên tiếp — ví dụ triage(true) rồi convert-to-training ngay sau đó. Tổng quát: N -switch = chuỗi $N+1$ chuyển tiếp liên tiếp.

0-switch (all transitions) cho PhishingReport

- ➊ submit → SUBMITTED
- ➋ SUBMITTED → CONFIRMED
- ➌ SUBMITTED → DISMISSED
- ➍ CONFIRMED → convert-to-training

+ Invalid transitions:

- ➎ triage lại report CONFIRMED
- ➏ convert report DISMISSED

Bảng test case chuyển trạng thái

TC-ID	Loại	Kịch bản	Expected
TC-ST-1	valid	POST /reports/phishing với payload mẫu	200, status submitted
TC-ST-2	valid	triage decision=confirm_threat report SUBMITTED	status confirmed, aiLabel threat, event published
TC-ST-3	valid	triage decision=dismiss report SUBMITTED	status dismissed, aiLabel clean, KHÔNG có event
TC-ST-4	valid	convert-to-training report CONFIRMED	convertedToTraining=true
TC-ST-5	invalid	triage dismiss report đã CONFIRMED	kỳ vọng bị từ chối — thực tế: đổi thành DISMISSED!
TC-ST-6	invalid	convert-to-training report DISMISSED	kỳ vọng bị từ chối — thực tế: vẫn set flag
TC-ST-7	invalid	triage id không tồn tại	lỗi “Không tìm thấy báo cáo”

TC-ST-5/6 **fail so với đặc tả kỳ vọng** — ghi nhận thành defect report, đem thảo luận với đội phát triển (bài học buổi 12 về quy trình defect).

Use Case Testing — Nguyên lý

Định nghĩa

Thiết kế test case từ **kịch bản sử dụng** (use case) mô tả cách người dùng tương tác với hệ thống để đạt một mục tiêu nghiệp vụ.

Cấu trúc một Use Case:

- **Actor**: người / hệ thống tương tác
- **Precondition**: điều kiện trước
- **Main flow**: luồng chính (happy path)
- **Alternative**: luồng thay thế
- **Exception**: luồng lỗi
- **Postcondition**: trạng thái sau

Test case được sinh từ:

- Một TC cho luồng chính
- Một TC cho mỗi luồng thay thế
- Một TC cho mỗi luồng lỗi
- TC khi precondition không thỏa

Khác EP/BVA: kiểm thử **chuỗi bước xuyên nhiều API**, phát hiện lỗi **tích hợp nghiệp vụ**.

UC-DS-01: Analyst xử lý báo cáo phishing

Main flow (pipeline AIDA)

Actor: Learner + Analyst

Precondition: đăng nhập; tenant demo

- 1 Learner nghi ngờ email, gửi POST /reports/phishing
- 2 Analyst mở SOC Inbox, GET /reports/phishing
- 3 Analyst xác nhận mối đe dọa: POST /reports/phishing/{id}/trriage
- 4 Hệ thống publish PhishingReportConfirmedEvent
- 5 Analytics chấm lại rủi ro — score của learner **tăng**
- 6 Learning **tự ghi danh** learner vào khóa nhận thức

Alternative / Exception flows

AF1: Analyst bác bỏ (decision=dismiss) → DISMISSED, **không** có bước 4–6

AF2: Analyst chuyển báo cáo thành bài huấn luyện (convert-to-training)

EF1: id không tồn tại → lỗi

EF2: báo cáo thuộc tenant khác → coi như không tồn tại (fail-closed)

EF3: role LEARNER gọi triage → 403; chưa đăng nhập → 401

Test case từ UC-DS-01

TC-ID	Kịch bản	Các bước chính	Expected
TC-UC-1	Main flow	submit → triage(true) → GET /analytics/risk	confirmed; risk tăng; enrollment mới
TC-UC-2	AF1	submit → triage(false)	dismissed; risk KHÔNG đổi
TC-UC-3	AF2	trriage(true) → convert-to-training	convertedToTraining=true
TC-UC-4	EF1	trriage với UUID ngẫu nhiên	lỗi, không đổi dữ liệu
TC-UC-5	EF2	trriage report của tenant khác	“không tìm thấy” (không lộ dữ liệu)
TC-UC-6	EF3	trriage bằng token role LEARNER (ngoài dev)	403 Forbidden
TC-UC-7	EF3	trriage không đăng nhập	401 Unauthorized

Nhận xét

TC-UC-1 xuyên qua **4 module** (reporting, analytics, learning, ai) — đúng loại lỗi mà unit test từng module không bao giờ thấy. Buổi 7 ta đã test pipeline này bằng @SpringBootTest; buổi 9 sẽ chạy nó bằng Postman.

Error Guessing — đoán lỗi bằng kinh nghiệm

Ý tưởng

Tester giàu kinh nghiệm **đoán** nơi dễ có lỗi dựa trên các dạng lỗi kinh điển, rồi thiết kế test nhắm thẳng vào đó. Bổ trợ — không thay thế — các kỹ thuật có hệ thống.

Checklist đoán lỗi áp lên DigiShield:

- Chuỗi rỗng / null / toàn khoảng trắng: email " " (3 dấu cách) qua được `isBlank()` không?
- Unicode, emoji, tiếng Việt có dấu trong payload báo cáo phishing
- Số âm, số 0, `Integer.MAX_VALUE` cho page, size
- Trùng lặp: gửi cùng một request hai lần (POST /users cùng email — DigiShield chọn upsert idempotent, có đúng ý spec?)
- Chuỗi rất dài (payload 1MB), ký tự đặc biệt SQL/HTML (' 0R 1=1, `<script>`)
- Sai kiểu JSON: "size": "abc", thiếu dấu ngoặc

Mẹo: giữ một **fault catalogue** của nhóm — mỗi defect tìm được thêm một dòng, dùng lại cho module sau.

Exploratory Testing — kiểm thử khám phá

Ý tưởng

Học — thiết kế — thực thi — đánh giá diễn ra **đồng thời**. Không có kịch bản viết trước; tester vừa dùng hệ thống vừa đào sâu theo những gì quan sát được.

Session-based: mỗi phiên 60–90 phút có:

- **Charter:** mục tiêu khám phá
- **Time-box:** giới hạn thời gian
- **Ghi chú:** bug, câu hỏi, ý tưởng test mới

Kết quả phiên khám phá thường **quay lại bổ sung** cho các kỹ thuật hệ thống: một quan sát lạ → một lớp tương đương mới hoặc một rule mới.

Charter mẫu cho DigiShield

“Khám phá SOC Inbox (/soc/inbox) với vai ANALYST: tìm cách làm nhãn tuổi, bộ lọc trạng thái và nút triage hiển thị sai hoặc mâu thuẫn nhau.”

Lựa chọn kỹ thuật phù hợp

Tình huống	Kỹ thuật	Ví dụ DigiShield
Một trường có phạm vi giá trị	EP + BVA	size 1..100, score 0..100, ageLabel
Trường kiểu enum	EP	action của /sim/events, role khi login
Nhiều điều kiện kết hợp	Decision Table	onCall + newPayee + watchlistHit
Hệ thống có trạng thái	State Transition	PhishingReport, AiTemplate, chiến dịch
Kịch bản người dùng xuyên module	Use Case	Pipeline triage → risk → enroll
Nghi ngờ theo kinh nghiệm	Error Guessing	email blank → 500, clamp lằng lẽ
Chưa hiểu rõ hệ thống	Exploratory	phiên khám phá SOC Inbox

Thực tế

Một bộ test case tốt **trộn nhiều kỹ thuật**: EP/BVA cho từng trường, Decision Table cho logic lỗi, Use Case cho luồng tổng thể, Error Guessing rắc thêm “gia vị”.

Bài tập 1 (nhóm): EP + BVA cho POST /auth/login

Đặc tả

POST /api/v1/auth/login với body {email, password} (record LoginRequest trong AuthController). Thành công trả về TokenPair. User seed: admin@coquan.gov.vn.

TC-ID	email	password	Kỹ thuật	Expected
TC-L1	admin@coquan.gov.vn	demo1234	EP-valid	200 + token
TC-L2	(rỗng)	demo1234	EP-invalid	?
TC-L3	...	...	...	...

(nhóm hoàn thiện tối thiểu 12 TC)

Yêu cầu

Phân lớp đủ 2 trường (email theo tồn tại + định dạng; password đúng/sai/rỗng) + trường thừa trong body. Chạy thử bằng curl/Postman để điền cột Expected **thực tế** — chú ý: dev dùng StubAuthProvider, password *không* được kiểm tra; lịch spec–hiện thực chính là phát hiện đáng ghi nhận.

Bài tập 2 (nhóm): Decision Table cho moderate ()

Đặc tả POST /ai/moderate (role ANALYST)

Kiểm duyệt nội dung mẫu phishing trước khi dùng cho đào tạo: không chứa từ không an toàn → "pass"; chứa **đúng 1** từ → "flag"; chứa **từ 2 từ trở lên** → "block". (Danh sách từ: malware, ransomware, exploit, ddos, ...)

Yêu cầu:

- 1 Đặt 2 điều kiện: $C1 = \text{"có } \geq 1 \text{ từ không an toàn"}$, $C2 = \text{"có } \geq 2 \text{ từ không an toàn"}$. Lập bảng đầy đủ $2^2 = 4$ rules.
- 2 Phát hiện rule **bất khả thi** (infeasible): $C1=F$, $C2=T$ — vì sao không thể xảy ra? Xử lý thế nào trong bảng?
- 3 Mỗi rule khả thi → một test case với content cụ thể (tự chọn từ trong danh sách).
- 4 Bonus (BVA trên số đếm): 0, 1, 2, 3 từ — biên nằm ở đâu?

Bài tập 3 (nhóm): chuyển trạng thái + tổng hợp

Yêu cầu

- 1 Vẽ lại state diagram cho PhishingReport **theo đề xuất của nhóm** (sửa các vấn đề đã phát hiện: TRIAGING, guard trạng thái).
- 2 Thiết kế bảng test 0-switch (all transitions): mọi transition hợp lệ + tối thiểu 3 transition không hợp lệ.
- 3 Gộp kết quả 3 bài tập thành một bảng test case chung.

Khuôn mẫu bảng dùng hôm nay (bản rút gọn)

TC-ID	Mục tiêu	Precondition	Input	Expected	Kỹ thuật
-------	----------	--------------	-------	----------	----------

Khuôn mẫu đặc tả test case **đầy đủ** (steps, priority, actual, trạng thái...) sẽ học ở **buổi 9** — hôm nay tập trung vào *chọn đúng ca test*.

Bài nộp — Deadline: trước buổi 9

Gắn với Bài tập lớn (A1.2 — 30%)

Mỗi nhóm nộp **bộ test case hộp đen cho module DigiShield của nhóm** (hoặc ứng dụng nhóm tự chọn đã được GV duyệt):

- 1 **Tối thiểu 20 test case**, dùng ít nhất **3 kỹ thuật** (EP/BVA bắt buộc; chọn thêm DT, ST hoặc UC tùy module).
- 2 Với mỗi kỹ thuật: kèm **sản phẩm trung gian** — bảng phân lớp, sơ đồ biên, decision table (đầy đủ + rút gọn), state diagram.
- 3 Đánh dấu các TC mà **Expected chưa xác định được do thiếu spec** — kèm câu hỏi gửi “product owner” (GV đóng vai).
- 4 Ít nhất **2 TC tiêu cực về phân quyền** (401/403).

Định dạng nộp

File PDF hoặc bảng tính, tên `Nhom_XX_Buoi08.pdf`. Bộ test case này sẽ được **thực thi bằng Postman ở buổi 9** và là một chương của báo cáo BTL cuối kỳ.

Câu hỏi ôn tập

- 1 EP và BVA khác nhau thế nào? Với size 1..100, liệt kê đầy đủ các điểm BVA 2-value.
- 2 DigiShield “clamp” size thay vì trả 400. Nêu một lập luận ủng hộ và một lập luận phản đối; tester nên làm gì?
- 3 Bảng quyết định đầy đủ của `evaluate()` có mấy rule? Vì sao rule (`onCall=T`, `newPayee=T`, `watchlistHit=F`) đáng đưa ra thảo luận với product owner?
- 4 Vì sao phải test cả **invalid transitions**? Cho ví dụ một invalid transition của `PhishingReport` mà code hiện tại **không** chặn.
- 5 Use Case Testing phát hiện loại lỗi gì mà EP/BVA bỏ sót? Minh họa bằng pipeline triage → risk → enroll.
- 6 Test hộp đen tìm ra lỗi “email rỗng trả 500”: đây là lỗi của code hay của hợp đồng API? Ai cần sửa gì?

Tóm tắt buổi 8

Kỹ thuật đã học:

- **EP**: phân lớp, chọn đại diện
- **BVA**: on-point / off-point tại biên
- **Decision Table**: tổ hợp điều kiện, rút gọn
- **State Transition**: valid + invalid transitions
- **Use Case**: kịch bản xuyên module
- Error Guessing, Exploratory (bổ trợ)

Đã áp dụng lên DigiShield:

- ageLabel, size, action
- evaluate() — bảng 8 rule
- PhishingReport, AiTemplate
- UC-DS-01: pipeline AIDA
- 2 defect/lỗi hỏng spec tìm được!

Buổi tới — Buổi 9

Khuôn mẫu đặc tả test case; API testing với **Postman + Newman**.
Chuẩn bị: cài Postman; mang theo bộ test case bài nộp.

Tài liệu tham khảo

- [1] Slide bài giảng điện tử môn Kiểm thử Phần mềm.
- [2] Paul Ammann, Jeff Offutt, *Introduction to Software Testing*, Cambridge University Press, 2008.
- [3] Glenford J. Myers, *The Art of Software Testing*, 2nd ed., Wiley, 2004 — đặc biệt Chương 4: *Test-Case Design*.
- [4] ISTQB — International Software Testing Qualifications Board, <https://istqb.org/>.
- [5] ISTQB Foundation Level Working Group, *Certified Tester Foundation Level Syllabus*, v4.0, 2023.
- [6] Mã nguồn và OpenAPI spec của DigiShield: `docs/DigiShield_openapi.yaml`, các module `interception`, `reporting`, `ai`, `auth`.